

Performance Testing

Date	1 July, 2026
Team ID	
Project Name	Smart Agricultural Production Optimization Engine (OptiCrop)
Maximum Marks	5 Marks

Step 1: Testing Overview

Field	Details
Testing Tool Used	Postman
Type of Testing	Load Testing
Target Module	Flask API (/predict_crop, /check_suitability), User Input
Test Environment	Local System (Windows 11, Python 3.11, Flask, ML Model)
Test Date	1 July, 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of virtual Users	Duration (sec)	Expected Outcome
1	Single user enters soil & environmental data for crop recommendation	1	30	Crop recommendation generated within 2 seconds
2	Multiple users requesting crop suitability analysis	100	120	Stable responses with no failures
3	Peak load testing with simultaneous API requests	200	180	System remains responsive and efficient

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (pass/fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.28 sec	Pass	Fast recommendation generation
2	Response Time (Max)	< 5 seconds	3.63 sec	Pass	Within acceptable limit
3	Throughput (Req/sec)	>40 req/sec	52 req/sec	Pass	Good system handling
4	Error Rate	<1%	0%	Pass	No failures observed
5	CPU Utilization	<80%	61%	Pass	Efficient processing

6	Memory Utilization	<80%	57%	Pass	Stable memor y usage
---	--------------------	------	-----	------	----------------------------

Step 4: Observations & Analysis

Key Findings

- The OptiCrop system successfully handled multiple concurrent user requests.
- Crop recommendation and suitability analysis responses remained under 2 seconds on average.
- No request failures were observed during testing.
- The Flask-based API maintained stability under both moderate and heavy loads.
- Machine learning model predictions remained consistent and efficient under stress.

Bottlenecks Identified

- Slight increase in response time when concurrent users exceeded 200.
- Initial delay observed during the first prediction due to model loading.
- Minor latency when processing large input combinations of environmental parameters.

Optimization Steps Taken

- Optimized API routing and reduced redundant computations.
- Preloaded the trained ML model during application startup.
- Minimized preprocessing overhead for input data.
- Used efficient NumPy/Pandas operations for faster predictions.
- Improved input validation to reduce unnecessary processing.